

Networks and Causal Inference Short Course: Part 3

TingFang Lee, Ashley Buchanan, URI Avenir Team

2026-04-30

Using Data from the Transmission Reduction Intervention Project in Athens, Greece

Download the data and load it into R Studio

This is a simulated toy dataset made for this course. It is based on data collected from the Transmission Reduction Intervention Project in Athens Greece. Public data here: <https://www.icpsr.umich.edu/web/NAHDAP/studies/39059/versions/V2/staff>

Read more: <https://pubmed.ncbi.nlm.nih.gov/27917890/>

Load packages and function script...

```
# -----  
# 1. Read data and functions into R  
# -----
```

```
library(lme4)  
library(plyr)  
library(dplyr)  
library(igraph)  
library(numDeriv)  
library(gtools)  
library(doParallel)  
library(foreach)  
library(knitr)
```

```
## Warning: package 'knitr' was built under R version 4.4.3
```

```
library(kableExtra)  
library(ggplot2)  
library(forcats)  
rm(list = ls())
```

```
if ("package:sna" %in% search()) {  
  detach("package:sna", unload = TRUE)  
} # sna package is used in part 2, but it will mess up with igraph function in part 3, so I unload it  
  
no_cores <- detectCores() - 1  
registerDoParallel(cores=no_cores)
```

```

#source("https://github.com/uri-ncipher/Nearest-Neighbor-estimators/blob/main/functions.R?raw=TRUE")
#source("IPW_functions.R")
#source("gcomp_dr_v2.R") #Ting-Fang removed warnings for bootstrap

# Source required R scripts directly from GitHub
# source("https://raw.githubusercontent.com/uri-ncipher/Causal-Inference-and-Networks-Short-Course/refs/heads/main/IPW_functions.R")
source("https://raw.githubusercontent.com/uri-ncipher/Causal-Inference-and-Networks-Short-Course/refs/heads/main/IPW_functions.R")

### load local IPW_functions.R in order to adding cut to IPW1 propensity score
source("./IPW_functions.R")

```

Data Preparation

Read in the synthetic nodes and edges data sets for creating the simulated network.

```

# -----
# 2. Data Preparation
# -----

nodes <- read.csv("https://raw.githubusercontent.com/uri-ncipher/Causal-Inference-and-Networks-Short-Course/refs/heads/main/nodes.csv")
edges <- read.csv("https://raw.githubusercontent.com/uri-ncipher/Causal-Inference-and-Networks-Short-Course/refs/heads/main/edges.csv")

head(nodes)

```

```

##      id num_nn component posneg date share education employment age avg_posneg
## 1  1      4          1      1    1      1          0          1  39  1.0000000
## 2  2      3          1      1    0      0          1          0  25  0.6666667
## 3  3      1          1      1    0      1          0          1  40  1.0000000
## 4  4      3          1      0    1      1          1          0  37  0.6666667
## 5  5      4          1      1    0      1          1          0  28  0.5000000
## 6  6      2          1      1    1      0          0          1  17  1.0000000
##      avg_date avg_share avg_education avg_employment avg_age      raneff treatment
## 1  0.5000000  0.5000000    0.7500000    0.5000000    27.75 -0.1141261          0
## 2  1.0000000  1.0000000    0.6666667    0.6666667    35.67 -0.1141261          0
## 3  0.0000000  1.0000000    0.0000000    1.0000000    31.00 -0.1141261          1
## 4  0.3333333  0.3333333    0.6666667    0.3333333    29.00 -0.1141261          0
## 5  0.7500000  0.5000000    0.5000000    0.2500000    36.75 -0.1141261          0
## 6  1.0000000  0.5000000    0.5000000    0.5000000    38.00 -0.1141261          1
##      num_tnn num_utnn outcome
## 1          0          4       1
## 2          0          3       0
## 3          0          1       1
## 4          0          3       0
## 5          1          3       1
## 6          1          1       0

```

```
head(edges)
```

```

##      from to
## 1      1  2
## 2      2  4

```

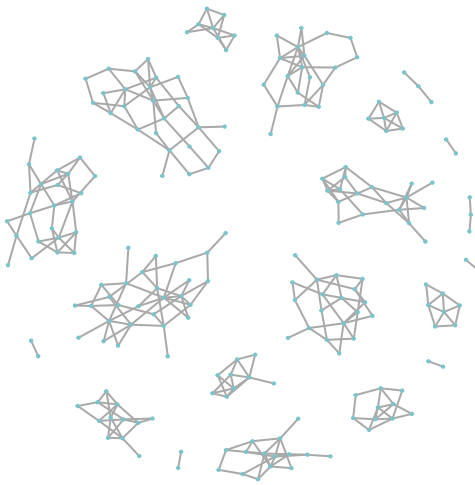
```
## 3    5    8
## 4    1    9
## 5    3    9
## 6    1   10
```

```
net0=graph_from_data_frame(d=edges, vertices = nodes, directed=F) # create network based on the nodes
```

Network visualization

```
# -----
# 3. Network Visualizaiton
# -----

# plot the network generated above
plot(net0, vertex.size=1, vertex.label=NA, vertex.color="cadetblue3", vertex.frame.color="cadetblue3")
```



Prepare the data for modeling

```
# -----
# 3. Data Preparation for Modeling
# -----
```

```

# Save the number of individuals (nodes) and number of components
n=length(V(net0))      # number of individuals
m=components(net0)$no  # number of components

# generate a dataset with three columns, the first column is participant id (node id),
# the second column is number of nearest neighbors for each participant (is also node degree),
# the third column indicate which component the participant is in.
df=data.frame(id=1:n, na=unlist(lapply(1:n, num_neighbors, net=net0)),
              component=components(net0)$membership)

# assign treatment, outcome, and baseline covariates to data
df$treatment=nodes$treatment
df$outcome=nodes$outcome
df$age=nodes$age
df$age10=nodes$age/10 #age per 10 years
df$agebin=ifelse(nodes$age>33,1,0) #age >33 years old
table(df$treatment, df$agebin)

##
##      0    1
##  0 82 106
##  1 15  13

df$posneg=nodes$posneg
df$date=nodes$date
df$share=nodes$share
df$education=nodes$education
df$employment=nodes$employment
df$na_a= unlist(lapply(1:n, trt_neighbors, net=net0))
df$notna_a=df$na-df$na_a

# six covariates, including age
base_covariate=c("age10", "posneg", "date", "share", "education", "employment")

# averaging baseline covariates (average of nearest neighbors' baseline covariate values)
# Note: show code for averaging "age10" and "agebin" as examples. Average of other neighbor covariates
#       Example code: df$avg_var1=unlist(lapply(1:n, avg_neighbors, net=net0, variable="var1")) # "var1"
df$avg_agebin=unlist(lapply(1:n, avg_neighbors, net=net0, variable="agebin"))
df$avg_age10 =unlist(lapply(1:n, avg_neighbors, net=net0, variable="age10"))

df$avg_posneg = nodes$avg_posneg
df$avg_date = nodes$avg_date
df$avg_share = nodes$avg_share
df$avg_education = nodes$avg_education
df$avg_employment = nodes$avg_employment

avg_covariate=c("avg_age10", "avg_posneg", "avg_date", "avg_share", "avg_education", "avg_employment")

```

Modeling

In this section, we will apply both IPW1 and IPW2 estimators for the average potential outcomes and corresponding causal effects. An individual's exposure/treatment and allocation strategy α jointly determine the

average potential outcome (allocation strategy represents the probability that individuals in nearest neighbors set receiving the exposure/treatment in the counterfactual scenario, so it is a numeric value between 0 and 1). For each estimator, the R programs will output the **point estimation and the estimated variance of average potential outcomes** $\hat{Y}(1, \alpha)$, $\hat{Y}(0, \alpha)$, $\hat{Y}(\alpha)$ under three different allocation strategies α (i.e., 0.25, 0.50, and 0.75), respectively. Also, the R program will output the **point estimation and estimated variance of four causal effects: Direct (DE), Indirect (IE), Total (TE), and Overall effect (OE)**.

Equations for calculating the four causal effects are:

- $\widehat{DE} = \hat{Y}(1, \alpha) - \hat{Y}(0, \alpha)$
- $\widehat{IE} = \hat{Y}(0, \alpha_0) - \hat{Y}(0, \alpha_1)$
- $\widehat{TE} = \hat{Y}(1, \alpha_0) - \hat{Y}(0, \alpha_1)$
- $\widehat{OE} = \hat{Y}(\alpha_0) - \hat{Y}(\alpha_1)$,

where α_0 , α_1 both represents allocation strategies, and $\alpha_0 \neq \alpha_1$ (Note: The associated paper compared the estimated average potential outcomes with α_1 to α_0 for the indirect, total, and overall effects. The code here compared the estimated average potential outcomes with α_0 to α_1 for the indirect, total, and overall effects). Users can set any values between 0 and 1 for α_0 and α_1 . If given α is a vector, then the causal contrasts will produce pairwise comparisons in the sequential order of the values in α . For example, if $\alpha = c(0.75, 0.25, 0.50)$, the contrasts for spillover effects will compare

- (i) $\alpha_0 = 0.75$ versus $\alpha_1 = 0.25$;
- (ii) $\alpha_0 = 0.75$ versus $\alpha_1 = 0.50$;
- (iii) $\alpha_0 = 0.25$ versus $\alpha_1 = 0.50$.

IPW1

Using IPW1 to estimate the average potential outcomes and causal effects under allocation strategies α .

```
# -----
# 4a. IPW1 Estimator
# -----

alpha=c(0.5, 0.4, 0.3, 0.2) # set allocation strategies

base_covariate=c("agebin", "posneg", "date", "share", "education", "employment")
avg_covariate=c("avg_agebin", "avg_posneg", "avg_date", "avg_share", "avg_education", "avg_employment")
#Note: Age variable continuous removed due to convergence issue, used binary for now

IPW1 <- IPW_1_model(df, base_covariate, alpha)

## [1] "summary(score):"
##      Min.   1st Qu.   Median     Mean   3rd Qu.     Max.
## 0.0004355 0.0857978 0.5219730 0.4342091 0.6918815 0.8869194

IPW1

## [[1]]
##      a=1      a=0      margin alpha      type
## 1 0.25529569 0.360633818 0.307964755 0.5 point estimate
```

```

## 2 0.35299392 0.409391168 0.386832268 0.4 point estimate
## 3 0.47297810 0.451569373 0.457991992 0.3 point estimate
## 4 0.60178904 0.480892919 0.505072143 0.2 point estimate
## 5 0.01197303 0.006377590 0.006241201 0.5 variance
## 6 0.02225333 0.006663655 0.008690831 0.4 variance
## 7 0.03733800 0.007352668 0.010751031 0.3 variance
## 8 0.05183687 0.008203765 0.011392133 0.2 variance
##
## [[2]]
##      estimation alpha0 alpha1      type
## 1  -0.1053381259    0.5    0.5  Direct
## 2  -0.0563972494    0.4    0.4  Direct
## 3   0.0214087314    0.3    0.3  Direct
## 4   0.1208961205    0.2    0.2  Direct
## 5   0.0117364409    0.5    0.5  Var DE
## 6   0.0175362249    0.4    0.4  Var DE
## 7   0.0266535111    0.3    0.3  Var DE
## 8   0.0346140798    0.2    0.2  Var DE
## 9  -0.0487573503    0.5    0.4 Indirect
## 10 -0.0909355550    0.5    0.3 Indirect
## 11 -0.1202591017    0.5    0.2 Indirect
## 12 -0.0421782047    0.4    0.3 Indirect
## 13 -0.0715017514    0.4    0.2 Indirect
## 14 -0.0293235467    0.3    0.2 Indirect
## 15  0.0004454532    0.5    0.4  Var IE
## 16  0.0017234579    0.5    0.3  Var IE
## 17  0.0035909238    0.5    0.2  Var IE
## 18  0.0004310698    0.4    0.3  Var IE
## 19  0.0015848385    0.4    0.2  Var IE
## 20  0.0003772808    0.3    0.2  Var IE
## 21 -0.1540954762    0.5    0.4  Total
## 22 -0.1962736808    0.5    0.3  Total
## 23 -0.2255972276    0.5    0.2  Total
## 24 -0.0985754541    0.4    0.3  Total
## 25 -0.1278990008    0.4    0.2  Total
## 26 -0.0079148153    0.3    0.2  Total
## 27  0.0097343132    0.5    0.4  Var TE
## 28  0.0084611152    0.5    0.3  Var TE
## 29  0.0080638593    0.5    0.2  Var TE
## 30  0.0155535556    0.4    0.3  Var TE
## 31  0.0146177626    0.4    0.2  Var TE
## 32  0.0251110409    0.3    0.2  Var TE
## 33 -0.0788675135    0.5    0.4 Overall
## 34 -0.1500272373    0.5    0.3 Overall
## 35 -0.1971073887    0.5    0.2 Overall
## 36 -0.0711597239    0.4    0.3 Overall
## 37 -0.1182398753    0.4    0.2 Overall
## 38 -0.0470801514    0.3    0.2 Overall
## 39  0.0003716931    0.5    0.4  Var OE
## 40  0.0012527940    0.5    0.3  Var OE
## 41  0.0023005983    0.5    0.2  Var OE
## 42  0.0002950416    0.4    0.3  Var OE
## 43  0.0010944169    0.4    0.2  Var OE
## 44  0.0003338455    0.3    0.2  Var OE

```

In the output above,

- The **section** `[[1]]` displays the point estimates (type = “point estimate” in the output) and estimated variances (type = “variance” in the output) for the average potential outcomes under three different allocation strategies (i.e., $\alpha = 0.25, 0.50, 0.75$) and each individual exposure ($a = 0, a = 1$, and margin). “Margin” is the average potential outcomes for a particular allocation strategy, regardless of individual exposure status.
- The **section** `[[2]]` displays the point estimates (type = “Direct”, “Indirect”, “Total”, “Overall” in the output) and estimated variances (type = “Var DE”, “Var IE”, “Var TE”, “Var OE” in the output) of four causal effects: direct, indirect, total and overall effects, corresponding to particular allocation strategies (α_0 and α_1). The estimated values are shown in “estimation” column in the output above.

Goodness of Fit test for random effect in mixed effect model.

```
### install glmerGOF package (github link: https://github.com/BarkleyBG/glmerGOF)
# remotes::install_github("BarkleyBG/glmerGOF")
```

```
library(glmerGOF)
library(survival)
library(cubature) # testGOF() function requires this package
```

```
## Warning: package 'cubature' was built under R version 4.4.3
```

```
### Fit a lme4::glmer() model (mixed effect model)
```

```
formula_glmer <- as.formula(paste("treatment ~", paste(base_covariate, collapse = "+"), "+ (1 | component)"))
formula_glmer
```

```
## treatment ~ agebin + posneg + date + share + education + employment +
##      (1 | component)
```

```
fit_glmer <- glmer(formula_glmer, data=df, family=binomial(link="logit"))
```

```
### Fit a survival::clogit() model
```

```
formula_clogit <- as.formula(paste("treatment ~", paste(base_covariate, collapse = "+"), "+ strata(component)"))
formula_clogit
```

```
## treatment ~ agebin + posneg + date + share + education + employment +
##      strata(component)
```

```
fit_clogit <- survival::clogit(formula_clogit, data=df, method="exact")
```

```
variable_names <- list(DV="treatment", grouping="component")
```

```
TC_test <- testGOF(data=df,
  fitted_model_clogit = fit_clogit,
  fitted_model_glmm = fit_glmer,
  var_names = variable_names,
  gradient_derivative_method = "simple"
)
```

```
TC_test$results
```

```
## $D
## [1] -5.604119
##
## $p_value
## [1] 1
##
## $df
## [1] 6
```

```
# ### Shapiro-Wilk normality test
# re <- ranef(fit_glmer)
# re_values <- re$component[["(Intercept)"]]
# shapiro.test(re_values)
#
# ### Lilliefors (Kolmogorov-Smirnov) test for normality
# library(nortest)
# nortest::lillie.test(re_values)
```

IPW2

Using IPW2 to estimate the average potential outcomes and causal effects under allocation strategies α .

```
# -----
# 4b. IPW2 Estimator
# -----
#change back to age per 10 year increase
base_covariate=c("age10", "posneg", "date", "share", "education", "employment")
avg_covariate=c("avg_age10", "avg_posneg", "avg_date", "avg_share", "avg_education", "avg_employment")

alpha=c(0.5, 0.4, 0.3, 0.2) # set allocation strategies

formula_1=paste("cbind(na_a, notna_a)", "~", "treatment", "+",
                paste(base_covariate, collapse = "+"), "+",
                paste(avg_covariate, collapse = "+"))
formula_2=paste("treatment", "~", paste(base_covariate, collapse = "+"))
M1=glm(formula_1, family = binomial(link = "logit"), data=df)
M2=glm(formula_2, family = binomial(link = "logit"), data=df)

IPW2 <- IPW_2_model(df, M1, M2, alpha)
IPW2
```

```
## [[1]]
##          a=1          a=0      margin alpha          type
## 1 0.220810727 0.59176096 0.40628585    0.5 point estimate
## 2 0.292392238 0.59042302 0.47121071    0.4 point estimate
## 3 0.387385956 0.57233698 0.51685167    0.3 point estimate
## 4 0.505408711 0.54206916 0.53473707    0.2 point estimate
## 5 0.006943618 0.09001969 0.03029339    0.5      variance
## 6 0.012269497 0.05617945 0.02906814    0.4      variance
## 7 0.023480049 0.02942577 0.02347725    0.3      variance
## 8 0.044905869 0.01507811 0.01736360    0.2      variance
##
## [[2]]
```


##	estimation	alpha0	alpha1	type
## 1	-0.3709502369	0.5	0.5	Direct
## 2	-0.2980307830	0.4	0.4	Direct
## 3	-0.1849510226	0.3	0.3	Direct
## 4	-0.0366604489	0.2	0.2	Direct
## 5	0.0727530662	0.5	0.5	Var DE
## 6	0.0397805488	0.4	0.4	Var DE
## 7	0.0198324068	0.3	0.3	Var DE
## 8	0.0230003806	0.2	0.2	Var DE
## 9	0.0013379431	0.5	0.4	Indirect
## 10	0.0194239855	0.5	0.3	Indirect
## 11	0.0496918036	0.5	0.2	Indirect
## 12	0.0180860424	0.4	0.3	Indirect
## 13	0.0483538604	0.4	0.2	Indirect
## 14	0.0302678181	0.3	0.2	Indirect
## 15	0.0050022015	0.5	0.4	Var IE
## 16	0.0222549084	0.5	0.3	Var IE
## 17	0.0485650418	0.5	0.2	Var IE
## 18	0.0061643281	0.4	0.3	Var IE
## 19	0.0224483697	0.4	0.2	Var IE
## 20	0.0050978758	0.3	0.2	Var IE
## 21	-0.3696122938	0.5	0.4	Total
## 22	-0.3515262514	0.5	0.3	Total
## 23	-0.3212584334	0.5	0.2	Total
## 24	-0.2799447407	0.4	0.3	Total
## 25	-0.2496769226	0.4	0.2	Total
## 26	-0.1546832045	0.3	0.2	Total
## 27	0.0412503517	0.5	0.4	Var TE
## 28	0.0175815581	0.5	0.3	Var TE
## 29	0.0064830895	0.5	0.2	Var TE
## 30	0.0170154604	0.4	0.3	Var TE
## 31	0.0069144403	0.4	0.2	Var TE
## 32	0.0110807476	0.3	0.2	Var TE
## 33	-0.0649248621	0.5	0.4	Overall
## 34	-0.1105658262	0.5	0.3	Overall
## 35	-0.1284512251	0.5	0.2	Overall
## 36	-0.0456409641	0.4	0.3	Overall
## 37	-0.0635263630	0.4	0.2	Overall
## 38	-0.0178853989	0.3	0.2	Overall
## 39	0.0004946305	0.5	0.4	Var OE
## 40	0.0032369653	0.5	0.3	Var OE
## 41	0.0100291441	0.5	0.2	Var OE
## 42	0.0013422311	0.4	0.3	Var OE
## 43	0.0067075875	0.4	0.2	Var OE
## 44	0.0021150165	0.3	0.2	Var OE

In the output above,

- The **section** `[[1]]` displays the point estimates (type = “point estimate” in the output) and estimated variances (type = “variance” in the output) for the average potential outcomes under three different allocation strategies (i.e., $\alpha = 0.75, 0.50, 0.25$) and each individual exposure ($a = 0, a = 1$, and margin). “Margin” is the average potential outcomes for a particular allocation strategy, regardless of individual exposure status.

- The **section** `[[2]]` displays the point estimates (type = “Direct”, “Indirect”, “Total”, “Overall” in the output) and estimated variances (type = “Var DE”, “Var IE”, “Var TE”, “Var OE” in the output) of four causal effects: direct, indirect, total and overall effects, corresponding to particular allocation strategies (α_0 and α_1). The estimated values are shown in “estimation” column in the output above.

Now, we provide example code for an outcome-model based approach, g-computation. We use assumptions similar to IPW2 regarding the covariates for confounding from an individual’s contacts in the network. We obtain 95% confidence intervals using a bootstrap on each individual and the summary information for their contacts’ exposures and covariates.

```
# -----
# 4c. G-computation Estimator (similar assumptions to IPW2)
# -----

#Obtain G-computation estimates with bootstrap CIs
#Uses similar assumptions for interference as IPW2
#Bootstrap unit is each individual and the information for their contacts

alpha <- c(0.5, 0.4, 0.3, 0.2)

res_gcomp <- gcomp_with_bootstrap(
  data = df,
  alpha = alpha,
  z_covariates = base_covariate,
  zn_covariates = avg_covariate,
  B = 1000,
  seed = 2026
)

res_gcomp$results
```

##	estimation	alpha0	alpha1	type	lower	upper
## 1	-0.2942459106	0.5	0.5	Direct	-0.50761492	0.06533357
## 2	-0.2293979865	0.4	0.4	Direct	-0.42669257	0.06419693
## 3	-0.1598010505	0.3	0.3	Direct	-0.33798759	0.07663089
## 4	-0.0854203130	0.2	0.2	Direct	-0.25944479	0.12368184
## 5	0.0216511692	0.5	0.5	Var DE	NA	NA
## 6	0.0164000766	0.4	0.4	Var DE	NA	NA
## 7	0.0120227666	0.3	0.3	Var DE	NA	NA
## 8	0.0091832561	0.2	0.2	Var DE	NA	NA
## 9	0.0200839175	0.5	0.4	Indirect	-0.01693391	0.04428758
## 10	0.0403768693	0.5	0.3	Indirect	-0.03420301	0.09193725
## 11	0.0608319406	0.5	0.2	Indirect	-0.05179040	0.14283959
## 12	0.0202929518	0.4	0.3	Indirect	-0.01726860	0.04778418
## 13	0.0407480231	0.4	0.2	Indirect	-0.03485546	0.09824659
## 14	0.0204550713	0.3	0.2	Indirect	-0.01758550	0.05084486
## 15	0.0002468110	0.5	0.4	Var IE	NA	NA
## 16	0.0010415511	0.5	0.3	Var IE	NA	NA
## 17	0.0024635162	0.5	0.2	Var IE	NA	NA
## 18	0.0002745770	0.4	0.3	Var IE	NA	NA
## 19	0.0011522778	0.4	0.2	Var IE	NA	NA
## 20	0.0003021185	0.3	0.2	Var IE	NA	NA
## 21	-0.2741619931	0.5	0.4	Total	-0.47240345	0.07597257

```
## 22 -0.2538690413    0.5    0.3    Total -0.43353718 0.09322211
## 23 -0.2334139700    0.5    0.2    Total -0.40302065 0.10170925
## 24 -0.2091050347    0.4    0.3    Total -0.39125631 0.07023090
## 25 -0.1886499634    0.4    0.2    Total -0.36128077 0.09155945
## 26 -0.1393459792    0.3    0.2    Total -0.30901130 0.08943311
## 27  0.0196262256    0.5    0.4    Var TE          NA          NA
## 28  0.0180032470    0.5    0.3    Var TE          NA          NA
## 29  0.0168711890    0.5    0.2    Var TE          NA          NA
## 30  0.0148275764    0.4    0.3    Var TE          NA          NA
## 31  0.0137482572    0.4    0.2    Var TE          NA          NA
## 32  0.0110057702    0.3    0.2    Var TE          NA          NA
## 33 -0.0352798432    0.5    0.4    Overall -0.05516482 0.02138685
## 34 -0.0588057709    0.5    0.3    Overall -0.10258781 0.04539049
## 35 -0.0692069521    0.5    0.2    Overall -0.13731253 0.07315086
## 36 -0.0235259276    0.4    0.3    Overall -0.04687556 0.02419022
## 37 -0.0339271089    0.4    0.2    Overall -0.08552364 0.05206213
## 38 -0.0104011812    0.3    0.2    Overall -0.03868068 0.02949115
## 39  0.0003810760    0.5    0.4    Var OE          NA          NA
## 40  0.0013757670    0.5    0.3    Var OE          NA          NA
## 41  0.0027839896    0.5    0.2    Var OE          NA          NA
## 42  0.0003201982    0.4    0.3    Var OE          NA          NA
## 43  0.0011822972    0.4    0.2    Var OE          NA          NA
## 44  0.0002889667    0.3    0.2    Var OE          NA          NA
```

Now, we provide example code for an doubly-robust approach, augmented IPW. We use assumptions similar to IPW2 regarding the covariates for confounding from an individual's contacts in the network. We obtain 95% confidence intervals using a bootstrap on each individual and the summary information for their contacts' exposures and covariates.

```
# -----
# 4d. Augmented IPW Estimator (doubly robust (DR1))
# -----

#Obtain DR1 Estimates with Bootstrap CIs
#Uses similar assumptions for interference as IPW2
#Bootstrap unit is each individual and the information for their contacts
alpha=c(0.5, 0.4, 0.3, 0.2) # set allocation strategies

res_dr <- dr_with_bootstrap(
  data = df,
  alpha = alpha,
  z_covariates = base_covariate,
  zn_covariates = avg_covariate,
  B = 1000,
  seed = 2026
)
res_dr$results
```

```
##      estimation alpha0 alpha1      type      lower      upper
## 1 -0.4455634514    0.5    0.5   Direct -0.93715545 -0.016765493
## 2 -0.3577497959    0.4    0.4   Direct -0.76750642  0.030082246
## 3 -0.2539181376    0.3    0.3   Direct -0.56140257  0.094975729
## 4 -0.1342450916    0.2    0.2   Direct -0.39851105  0.232588637
```

## 5	0.0531417912	0.5	0.5	Var DE	NA	NA
## 6	0.0414580541	0.4	0.4	Var DE	NA	NA
## 7	0.0390018854	0.3	0.3	Var DE	NA	NA
## 8	0.0526416964	0.2	0.2	Var DE	NA	NA
## 9	0.0449502665	0.5	0.4	Indirect	-0.01084133	0.149703682
## 10	0.0918239271	0.5	0.3	Indirect	-0.02200590	0.315064980
## 11	0.1374273586	0.5	0.2	Indirect	-0.03276185	0.478209199
## 12	0.0468736607	0.4	0.3	Indirect	-0.01165771	0.167767222
## 13	0.0924770921	0.4	0.2	Indirect	-0.02306165	0.330362955
## 14	0.0456034314	0.3	0.2	Indirect	-0.01098028	0.160935258
## 15	0.0016549408	0.5	0.4	Var IE	NA	NA
## 16	0.0074018887	0.5	0.3	Var IE	NA	NA
## 17	0.0165648912	0.5	0.2	Var IE	NA	NA
## 18	0.0020647643	0.4	0.3	Var IE	NA	NA
## 19	0.0077881332	0.4	0.2	Var IE	NA	NA
## 20	0.0018424493	0.3	0.2	Var IE	NA	NA
## 21	-0.4006131849	0.5	0.4	Total	-0.80526016	-0.008135616
## 22	-0.3537395242	0.5	0.3	Total	-0.66760483	0.031608331
## 23	-0.3081360928	0.5	0.2	Total	-0.55378513	0.071303381
## 24	-0.3108761352	0.4	0.3	Total	-0.61707076	0.047007159
## 25	-0.2652727037	0.4	0.2	Total	-0.49385115	0.083106573
## 26	-0.2083147061	0.3	0.2	Total	-0.44164396	0.123325407
## 27	0.0404582776	0.5	0.4	Var TE	NA	NA
## 28	0.0301274437	0.5	0.3	Var TE	NA	NA
## 29	0.0240662094	0.5	0.2	Var TE	NA	NA
## 30	0.0309960585	0.4	0.3	Var TE	NA	NA
## 31	0.0248090126	0.4	0.2	Var TE	NA	NA
## 32	0.0326468813	0.3	0.2	Var TE	NA	NA
## 33	-0.0347315409	0.5	0.4	Overall	-0.06821944	0.023101619
## 34	-0.0547823573	0.5	0.3	Overall	-0.12702281	0.066721805
## 35	-0.0585053488	0.5	0.2	Overall	-0.16705565	0.146756561
## 36	-0.0200508164	0.4	0.3	Overall	-0.05813752	0.051277125
## 37	-0.0237738079	0.4	0.2	Overall	-0.09998771	0.114006996
## 38	-0.0037229915	0.3	0.2	Overall	-0.04554477	0.070645016
## 39	0.0006677682	0.5	0.4	Var OE	NA	NA
## 40	0.0026659983	0.5	0.3	Var OE	NA	NA
## 41	0.0059769886	0.5	0.2	Var OE	NA	NA
## 42	0.0007509683	0.4	0.3	Var OE	NA	NA
## 43	0.0030976513	0.4	0.2	Var OE	NA	NA
## 44	0.0008663912	0.3	0.2	Var OE	NA	NA

Lastly, we perform assessment of the propensity score in IPW2 in the presence of interference.

```
# -----
# 5. Assessing the fit of the generalized propensity scores
# -----

### Check covariate balance for generalized propensity score: Fit a model on each covariate, then include

# IPW2 was fitted using "age10", so we check balance using "age10" (continuous covariate), not "agebin"
base_covariate=c("age10", "posneg", "date", "share", "education", "employment")
avg_covariate=c("avg_age10", "avg_posneg", "avg_date", "avg_share", "avg_education", "avg_employment")
```

```

##### check propensity score balance for IPW2
mat_1 = as.matrix(cbind(1, df[, row.names(coef(summary(M1)))[-1]])) # neighbors' treatment
mat_2 = as.matrix(cbind(1, df[, row.names(coef(summary(M2)))[-1]])) # Individual treatment

group_ps <- plogis(mat_1 %*% coef(M1))
ind_ps <- plogis(mat_2 %*% coef(M2))
df$group_ps_ipw2 <- group_ps
df$ind_ps_ipw2 <- ind_ps #Ke, should this be 1-ps for those not exposed?

### Step 1: check individual-level propensity score balance
# Note: Compare two models w/ and w/o treatment:
#      (1) ind_covariate ~ ind_treatment + ind_ps_ipw2;
#      (2) ind_covariate ~ ind_ps_ipw2
ipw2_balance <- data.frame()
for (i in 1:length(base_covariate)) {
  f1 <- paste0(base_covariate[i], " ~ ind_ps_ipw2")
  f2 <- paste0(base_covariate[i], " ~ treatment + ind_ps_ipw2")

  if (base_covariate[i] == "age10"){
    model_reduced <- glm(f1, data = df, family = gaussian)
    model_full <- glm(f2, data = df, family = gaussian("identity"))
  } else {
    model_reduced <- glm(f1, data = df, family = binomial("logit"))
    model_full <- glm(f2, data = df, family = binomial("logit"))
  }
  res_temp <- lmtest::lrtest(model_reduced, model_full)

  ipw2_balance <- rbind(ipw2_balance,
                        c(f1, f2, round(res_temp$Chisq[2], 4), round(res_temp$`Pr(>Chisq)`[2], 4))
  )
}

### Step 2: check group-level propensity score balance
# Note: Compare two models w/ and w/o treatment:
#      (1) group_covariate ~ group_treatment + group_ps;
#      (2) group_covariate ~ group_ps.
for (i in 1:length(avg_covariate)) {
  f1 <- paste0(avg_covariate[i], " ~ group_ps_ipw2")
  f2 <- paste0(avg_covariate[i], " ~ cbind(na_a, notna_a) + group_ps_ipw2")

  model_reduced <- glm(f1, data = df)
  model_full <- glm(f2, data = df)

  res_temp <- lmtest::lrtest(model_reduced, model_full)

  ipw2_balance <- rbind(ipw2_balance,
                        c(f1, f2, round(res_temp$Chisq[2], 4), round(res_temp$`Pr(>Chisq)`[2], 4))
  )
}
colnames(ipw2_balance) <- c("Reduced model", "Full model", "Chisq", "Pr(>Chisq)")
ipw2_balance

```

```
## Reduced model
## 1 age10 ~ ind_ps_ipw2
## 2 posneg ~ ind_ps_ipw2
## 3 date ~ ind_ps_ipw2
## 4 share ~ ind_ps_ipw2
## 5 education ~ ind_ps_ipw2
## 6 employment ~ ind_ps_ipw2
## 7 avg_age10 ~ group_ps_ipw2
## 8 avg_posneg ~ group_ps_ipw2
## 9 avg_date ~ group_ps_ipw2
## 10 avg_share ~ group_ps_ipw2
## 11 avg_education ~ group_ps_ipw2
## 12 avg_employment ~ group_ps_ipw2
## Full model Chisq Pr(>Chisq)
## 1 age10 ~ treatment + ind_ps_ipw2 0.0141 0.9053
## 2 posneg ~ treatment + ind_ps_ipw2 5e-04 0.9815
## 3 date ~ treatment + ind_ps_ipw2 3e-04 0.9873
## 4 share ~ treatment + ind_ps_ipw2 0.0314 0.8593
## 5 education ~ treatment + ind_ps_ipw2 0.0015 0.9687
## 6 employment ~ treatment + ind_ps_ipw2 0.0055 0.941
## 7 avg_age10 ~ cbind(na_a, notna_a) + group_ps_ipw2 0.8063 0.6682
## 8 avg_posneg ~ cbind(na_a, notna_a) + group_ps_ipw2 0.5856 0.7462
## 9 avg_date ~ cbind(na_a, notna_a) + group_ps_ipw2 1.3213 0.5165
## 10 avg_share ~ cbind(na_a, notna_a) + group_ps_ipw2 5.956 0.0509
## 11 avg_education ~ cbind(na_a, notna_a) + group_ps_ipw2 0.3799 0.827
## 12 avg_employment ~ cbind(na_a, notna_a) + group_ps_ipw2 2.3496 0.3089
```

#Optional: Format Latex Table for output of each estimation approach

#Create forest plot

```
library(dplyr)
library(forcats)
library(ggplot2)
library(knitr)
library(kableExtra)
```

```
z <- qnorm(0.975)
```

```
#####
```

```
# 1. Result object helper
```

```
#####
```

```
#library(conflicted)
```

```
#conflicts_prefer(dplyr::filter) # filter will now always use dplyr, even if MASS is loaded
```

```
#conflicts_prefer(dplyr::count)
```

```
make_result_object <- function(x) {
  if (is.list(x) && !is.data.frame(x)) {
    df_idx <- which(sapply(x, function(y) {
      is.data.frame(y) &&
      all(c("estimation", "alpha0", "alpha1", "type") %in% names(y))
    }))
  }

  if (length(df_idx) == 0) {
    stop("No data frame with columns estimation, alpha0, alpha1, and type was found.")
  }
}
```

```

  x <- x[[df_idx[1]]]
}

if (!all(c("estimation", "alpha0", "alpha1", "type") %in% names(x))) {
  stop("Input must contain columns: estimation, alpha0, alpha1, type.")
}

est_rows <- x %>%
  filter(!grepl("^Var", type)) %>%
  mutate(
    effect_key = case_when(
      type == "Direct" ~ "DE",
      type == "Indirect" ~ "IE",
      type == "Total" ~ "TE",
      type == "Overall" ~ "OE",
      TRUE ~ type
    )
  )

var_rows <- x %>%
  filter(grepl("^Var", type)) %>%
  mutate(
    effect_key = gsub("^Var\\s+", "", type),
    variance = estimation
  ) %>%
  select(alpha0, alpha1, effect_key, variance)

results <- est_rows %>%
  left_join(
    var_rows,
    by = c("alpha0", "alpha1", "effect_key")
  ) %>%
  mutate(
    se = sqrt(variance),
    lower = estimation - z * se,
    upper = estimation + z * se
  ) %>%
  select(estimation, alpha0, alpha1, type, lower, upper)

list(results = results)
}

#####
# 2. Standardize alpha display by estimator/effect
#####

prep_effect_results <- function(results, estimator_name) {
  out <- results %>%
    filter(!grepl("^Var", type)) %>%
    mutate(
      estimator = estimator_name,
      effect = case_when(
        type == "Direct" ~ "Direct",

```

```

    type == "Indirect" ~ "Indirect",
    type == "Total" ~ "Total",
    type == "Overall" ~ "Overall",
    TRUE ~ type
  )
)

# G-comp and DR store coverage columns reversed relative to display.
if (estimator_name %in% c("G-comp", "DR")) {
  out <- out %>%
    mutate(
      raw_alpha1 = alpha0,
      raw_alpha0 = alpha1
    )
} else {
  out <- out %>%
    mutate(
      raw_alpha1 = alpha1,
      raw_alpha0 = alpha0
    )
}

out %>%
  mutate(
    display_alpha1 = pmax(raw_alpha1, raw_alpha0),
    display_alpha0 = pmin(raw_alpha1, raw_alpha0),
    contrast = paste0("(", display_alpha1, ", ", display_alpha0, ")")
  ) %>%
  filter(
    case_when(
      effect == "Direct" ~ display_alpha1 == display_alpha0,
      effect %in% c("Indirect", "Total") ~ display_alpha1 > display_alpha0,
      effect == "Overall" ~ TRUE,
      TRUE ~ TRUE
    )
  )
}

#####
# 3. LaTeX table helper
#####

make_latex_effect_table <- function(results,
                                     estimator_name,
                                     caption = "Estimated causal effects",
                                     digits = 3) {
  table_df <- prep_effect_results(results, estimator_name) %>%
    mutate(
      effect = factor(effect, levels = c("Direct", "Indirect", "Total", "Overall")),
      estimate = round(estimation, digits),
      ci = ifelse(
        is.na(lower) | is.na(upper),
        "",

```



```

      paste0(
        "(", round(lower, digits), ", ",
        round(upper, digits), ")"
      )
    )
  ) %>%
  arrange(effect, desc(display_alpha1), desc(display_alpha0)) %>%
  select(
    Effect = effect,
    `$(\\alpha_1, \\alpha_0)$` = contrast,
    `Point estimate` = estimate,
    `95\\% CI` = ci
  )

direct_n <- sum(table_df$Effect == "Direct")
indirect_n <- sum(table_df$Effect == "Indirect")
total_n <- sum(table_df$Effect == "Total")
overall_n <- sum(table_df$Effect == "Overall")

tab <- table_df %>%
  kable(
    format = "latex",
    booktabs = TRUE,
    escape = FALSE,
    caption = caption,
    align = c("l", "c", "c", "c")
  ) %>%
  kable_styling(latex_options = c("hold_position"))

start <- 1

if (direct_n > 0) {
  tab <- tab %>% pack_rows("Direct effects", start, start + direct_n - 1, italic = TRUE)
  start <- start + direct_n
}

if (indirect_n > 0) {
  tab <- tab %>% pack_rows("Indirect effects", start, start + indirect_n - 1, italic = TRUE)
  start <- start + indirect_n
}

if (total_n > 0) {
  tab <- tab %>% pack_rows("Total effects", start, start + total_n - 1, italic = TRUE)
  start <- start + total_n
}

if (overall_n > 0) {
  tab <- tab %>% pack_rows("Overall effects", start, start + overall_n - 1, italic = TRUE)
}

tab
}

```

```
#####
# 4. Create result objects
#####

res_ipw1 <- make_result_object(IPW1)
res_ipw2 <- make_result_object(IPW2)

res_gcomp <- if (exists("res_gcomp") && "results" %in% names(res_gcomp)) {
  res_gcomp
} else {
  make_result_object(gcomp_table)
}

res_dr <- if (exists("res_dr") && "results" %in% names(res_dr)) {
  res_dr
} else {
  make_result_object(dr_table)
}

#####
# 5. Create LaTeX tables
#####

ipw1_tab <- make_latex_effect_table(
  res_ipw1$results,
  estimator_name = "IPW1",
  caption = "IPW1 estimates of causal effects"
)

ipw2_tab <- make_latex_effect_table(
  res_ipw2$results,
  estimator_name = "IPW2",
  caption = "IPW2 estimates of causal effects"
)

gcomp_tab <- make_latex_effect_table(
  res_gcomp$results,
  estimator_name = "G-comp",
  caption = "G-computation estimates of causal effects"
)

dr_tab <- make_latex_effect_table(
  res_dr$results,
  estimator_name = "DR",
  caption = "Doubly robust estimates of causal effects"
)

cat(ipw1_tab, file = "ipw1_effects_table.tex")
cat(ipw2_tab, file = "ipw2_effects_table.tex")
cat(gcomp_tab, file = "gcomp_effects_table.tex")
cat(dr_tab, file = "dr_effects_table.tex")

#####
```

```

# 6. Forest plot data
#####

estimator_levels <- c("IPW1", "IPW2", "G-comp", "DR")

df <- bind_rows(
  prep_effect_results(res_ipw1$results, "IPW1"),
  prep_effect_results(res_ipw2$results, "IPW2"),
  prep_effect_results(res_gcomp$results, "G-comp"),
  prep_effect_results(res_dr$results, "DR")
) %>%
  mutate(
    estimator = factor(estimator, levels = estimator_levels),
    effect = factor(effect, levels = c("Direct", "Indirect", "Total", "Overall"))
  )

# Optional check: verify all estimator/effect/contrast combinations are present.
df %>%
  dplyr::count(estimator, effect, contrast) %>%
  arrange(estimator, effect, contrast)

```

```

##      estimator    effect  contrast n
## 1      IPW1    Direct (0.2, 0.2) 1
## 2      IPW1    Direct (0.3, 0.3) 1
## 3      IPW1    Direct (0.4, 0.4) 1
## 4      IPW1    Direct (0.5, 0.5) 1
## 5      IPW1 Indirect (0.3, 0.2) 1
## 6      IPW1 Indirect (0.4, 0.2) 1
## 7      IPW1 Indirect (0.4, 0.3) 1
## 8      IPW1 Indirect (0.5, 0.2) 1
## 9      IPW1 Indirect (0.5, 0.3) 1
## 10     IPW1 Indirect (0.5, 0.4) 1
## 11     IPW1      Total (0.3, 0.2) 1
## 12     IPW1      Total (0.4, 0.2) 1
## 13     IPW1      Total (0.4, 0.3) 1
## 14     IPW1      Total (0.5, 0.2) 1
## 15     IPW1      Total (0.5, 0.3) 1
## 16     IPW1      Total (0.5, 0.4) 1
## 17     IPW1 Overall (0.3, 0.2) 1
## 18     IPW1 Overall (0.4, 0.2) 1
## 19     IPW1 Overall (0.4, 0.3) 1
## 20     IPW1 Overall (0.5, 0.2) 1
## 21     IPW1 Overall (0.5, 0.3) 1
## 22     IPW1 Overall (0.5, 0.4) 1
## 23     IPW2    Direct (0.2, 0.2) 1
## 24     IPW2    Direct (0.3, 0.3) 1
## 25     IPW2    Direct (0.4, 0.4) 1
## 26     IPW2    Direct (0.5, 0.5) 1
## 27     IPW2 Indirect (0.3, 0.2) 1
## 28     IPW2 Indirect (0.4, 0.2) 1
## 29     IPW2 Indirect (0.4, 0.3) 1
## 30     IPW2 Indirect (0.5, 0.2) 1
## 31     IPW2 Indirect (0.5, 0.3) 1

```

```

## 32      IPW2 Indirect (0.5, 0.4) 1
## 33      IPW2   Total (0.3, 0.2) 1
## 34      IPW2   Total (0.4, 0.2) 1
## 35      IPW2   Total (0.4, 0.3) 1
## 36      IPW2   Total (0.5, 0.2) 1
## 37      IPW2   Total (0.5, 0.3) 1
## 38      IPW2   Total (0.5, 0.4) 1
## 39      IPW2 Overall (0.3, 0.2) 1
## 40      IPW2 Overall (0.4, 0.2) 1
## 41      IPW2 Overall (0.4, 0.3) 1
## 42      IPW2 Overall (0.5, 0.2) 1
## 43      IPW2 Overall (0.5, 0.3) 1
## 44      IPW2 Overall (0.5, 0.4) 1
## 45      G-comp Direct (0.2, 0.2) 1
## 46      G-comp Direct (0.3, 0.3) 1
## 47      G-comp Direct (0.4, 0.4) 1
## 48      G-comp Direct (0.5, 0.5) 1
## 49      G-comp Indirect (0.3, 0.2) 1
## 50      G-comp Indirect (0.4, 0.2) 1
## 51      G-comp Indirect (0.4, 0.3) 1
## 52      G-comp Indirect (0.5, 0.2) 1
## 53      G-comp Indirect (0.5, 0.3) 1
## 54      G-comp Indirect (0.5, 0.4) 1
## 55      G-comp   Total (0.3, 0.2) 1
## 56      G-comp   Total (0.4, 0.2) 1
## 57      G-comp   Total (0.4, 0.3) 1
## 58      G-comp   Total (0.5, 0.2) 1
## 59      G-comp   Total (0.5, 0.3) 1
## 60      G-comp   Total (0.5, 0.4) 1
## 61      G-comp Overall (0.3, 0.2) 1
## 62      G-comp Overall (0.4, 0.2) 1
## 63      G-comp Overall (0.4, 0.3) 1
## 64      G-comp Overall (0.5, 0.2) 1
## 65      G-comp Overall (0.5, 0.3) 1
## 66      G-comp Overall (0.5, 0.4) 1
## 67      DR   Direct (0.2, 0.2) 1
## 68      DR   Direct (0.3, 0.3) 1
## 69      DR   Direct (0.4, 0.4) 1
## 70      DR   Direct (0.5, 0.5) 1
## 71      DR Indirect (0.3, 0.2) 1
## 72      DR Indirect (0.4, 0.2) 1
## 73      DR Indirect (0.4, 0.3) 1
## 74      DR Indirect (0.5, 0.2) 1
## 75      DR Indirect (0.5, 0.3) 1
## 76      DR Indirect (0.5, 0.4) 1
## 77      DR   Total (0.3, 0.2) 1
## 78      DR   Total (0.4, 0.2) 1
## 79      DR   Total (0.4, 0.3) 1
## 80      DR   Total (0.5, 0.2) 1
## 81      DR   Total (0.5, 0.3) 1
## 82      DR   Total (0.5, 0.4) 1
## 83      DR Overall (0.3, 0.2) 1
## 84      DR Overall (0.4, 0.2) 1
## 85      DR Overall (0.4, 0.3) 1

```

```

## 86      DR Overall (0.5, 0.2) 1
## 87      DR Overall (0.5, 0.3) 1
## 88      DR Overall (0.5, 0.4) 1

#####
# 7. Forest plot helper
#####

estimator_colors <- c(
  "IPW1" = "#009E73",
  "IPW2" = "#0072B2",
  "G-comp" = "#56B4E9",
  "DR" = "#E69F00"
)

estimator_shapes <- c(
  "IPW1" = 15,
  "IPW2" = 16,
  "G-comp" = 17,
  "DR" = 18
)

estimator_offsets <- c(
  "IPW1" = 0.27,
  "IPW2" = 0.09,
  "G-comp" = -0.09,
  "DR" = -0.27
)

### specify x-axis range in the plot
x_min <- min(df$lower) - 0.5
x_max <- max(df$upper) + 0.1

make_effect_forest_plot <- function(plot_data, effect_name) {
  plot_data <- plot_data %>%
    filter(effect == effect_name)

  contrast_map <- plot_data %>%
    distinct(contrast, display_alpha1, display_alpha0) %>%
    arrange(desc(display_alpha1), desc(display_alpha0)) %>%
    dplyr::mutate(contrast_id = row_number())

  plot_data <- plot_data %>%
    left_join(
      contrast_map,
      by = c("contrast", "display_alpha1", "display_alpha0")
    ) %>%
    dplyr::mutate(
      y_pos = contrast_id + estimator_offsets[as.character(estimator)]
    )

  ggplot(
    plot_data,
    aes(color = estimator, shape = estimator)
  )
}

```

```

) +
  geom_vline(
    xintercept = 0,
    linetype = "dashed",
    linewidth = 0.6,
    color = "gray30"
  ) +
  geom_segment(
    aes(x = lower, xend = upper, y = y_pos, yend = y_pos),
    linewidth = 0.8
  ) +
  geom_point(
    aes(x = estimation, y = y_pos),
    size = 2.2
  ) +
  scale_x_continuous(limits=c(x_min, x_max)) +
  scale_y_continuous(
    breaks = contrast_map$contrast_id,
    labels = contrast_map$contrast
  ) +
  scale_color_manual(values = estimator_colors, breaks = estimator_levels) +
  scale_shape_manual(values = estimator_shapes, breaks = estimator_levels) +
  labs(
    title = paste(effect_name, "Effects"),
    x = "Estimate (95% CI)",
    y = expression(paste("Allocation (", alpha[1], ", ", alpha[0], ")")),
    color = "Estimator",
    shape = "Estimator"
  ) +
  theme_minimal(base_size = 16) +
  theme(
    legend.position = "top",
    plot.title = element_text(face = "bold", hjust = 0),
    axis.text.y = element_text(size = 12),
    panel.grid.minor = element_blank(),
    panel.grid.major.y = element_blank()
  )
}

#####
# 8. Create four forest plots
#####

p_direct <- make_effect_forest_plot(df, "Direct")
p_indirect <- make_effect_forest_plot(df, "Indirect")
p_total <- make_effect_forest_plot(df, "Total")
p_overall <- make_effect_forest_plot(df, "Overall")

ggsave("spillover_forest_direct.pdf", p_direct, width = 10, height = 4.5)
ggsave("spillover_forest_indirect.pdf", p_indirect, width = 10, height = 5.5)
ggsave("spillover_forest_total.pdf", p_total, width = 10, height = 5.5)
ggsave("spillover_forest_overall.pdf", p_overall, width = 10, height = 5.5)

```